HyperVista Windows Build Guide: From Build Kit to .exe

Nov 11, 2026 · @Kapih

Overview

This guide takes you from the HyperVista (DS3) Python file to a Windows program that anyone can start by double-clicking `HyperVista.exe`, with no Python installed. The program is a folder, not a single file: `HyperVista.exe` plus an `_internal` folder that holds Python and every library. The tool that does the packaging is PyInstaller, and it has to run on Windows to produce a Windows program.

The whole path, in order:

1. Write the build kit: the app file, a PyInstaller recipe (`HyperVista.spec`), package lists, a one-click script and an optional installer script.
2. Install Python 3.11 on a Windows PC and unzip the kit.
3. Run `build_windows.bat`. It creates a private Python environment, installs the packages, runs PyInstaller and tests the result.
4. Open `dist\HyperVista\HyperVista.exe` and check it starts.
5. Zip the `dist\HyperVista` folder to share it, or compile `installer.iss` into a single `HyperVista_Setup.exe`.

Build	Includes	Folder size	Use when
Lite (default)	Everything except TensorFlow	about 0.5 to 1 GB	Viewing, preprocessing, PCA, MNF, Random Forest and SVM, spectral matching, unmixing
Full	Lite plus TensorFlow and Keras	about 2 to 3 GB	You also need the 16 deep-learning Segmentation models
Debug	Lite with a console window	about 0.5 to 1 GB	The program closes by itself and you need to read the error

The build kit

The kit is the zip `HyperVista_build.zip`. Everything the build needs is inside it, so nothing has to be downloaded by hand except Python itself.

File	What it does
`HyperVista_v1_0.py`	The app. It is your v1.0a file plus three build-only additions: a `--selftest` option, messages that say "use the full build" when TensorFlow is missing in a packaged app, and a guard so the program runs without a console window
`HyperVista.spec`	The PyInstaller recipe: which file to start from, which data files and hidden modules to include, which to leave out, window or console mode, icon
`build_windows.bat`	The one-click script. Creates the environment, installs packages, builds and self-tests
`requirements.txt`	Packages for the lite build: PyQt5, numpy, scipy, matplotlib, scikit-learn, scikit-image, joblib, psutil, spectral, h5py, tifffile, netCDF4, astropy, zarr, PyInstaller
`requirements_tf.txt`	Extra packages for the full build: tensorflow-cpu, tf-keras, keras-cv-attention-models
`ds3.ico`, `hypervista.ico`	The same icon under two names, because the app looks for either
`installer.iss`	Optional Inno Setup script that wraps the finished folder into one installer
`.github/workflows/build-windows.yml`	Optional: lets GitHub build the Windows zip for you
`README_RELEASE.md`, `BUILD_ON_GITHUB.md`, `HOW_TO_BUILD_EXE.md`	Short user and build notes

How the kit was made

1. **Start from your app.** The v1.0a file was compared with the working copy. The only differences were the three build additions above.
2. **Write the recipe.** The app loads many readers by name at run time (h5py, tifffile, netCDF4, astropy, zarr, spectral, rasterio), which PyInstaller cannot see by itself. The spec lists them as hidden imports and bundles them when installed. It also excludes things the app never uses (Tk, PyQt6, PySide, torch, IPython) to keep the folder smaller.
3. **Split lite and full.** TensorFlow is imported lazily and is very large, so it is switched on only when the environment variable `HV_WITH_TF=1` is set (`build_windows.bat full` does this).
4. **Add a self-test.** `HyperVista.exe --selftest` starts the app off-screen, builds a synthetic cube, runs PCA, MNF, Random Forest and both SVMs, imports the optional libraries, draws a figure and writes `selftest.log`. It exits with an error code if anything essential fails. A missing TensorFlow does not count as a failure in the lite build.
5. **Test the recipe.** Because the sandbox is Linux, the same spec was built there in lite and full mode. Both folders started and passed the self-test (lite about 0.8 GB, full about 2.8 GB). A Windows build has not been run by me, so your first Windows build is the real test.

Create the build kit yourself

A build kit is an ordinary folder. PyInstaller needs four things from it: your app file, a recipe (the spec), a list of packages to install, and a script that runs the steps in the right order. This part shows how to make each one and why.

Step 1: make the folder and gather the files

1. Create a folder, for example `HyperVista_build`.
2. Copy your app into it as `HyperVista_v1_0.py`.
3. Copy the icon into it. The app looks for `hypervista.ico` and `ds3.ico`, so keep both names (they can be the same file).
4. Keep the folder name and path free of spaces and accents. A short path such as `C:\HyperVista_build` avoids Windows path-length problems.

Step 2: list the packages (the requirements files)

PyInstaller can only bundle what is installed, so the kit must say what to install. Find the list by reading the `import` lines at the top of the app, and by searching the file for modules loaded by name (for HyperVista, a helper that tries to import `h5py`, `tifffile`, `netCDF4`, `astropy`, `zarr`, `spectral` and similar).

`requirements.txt` (the lite build):

```
PyQt5>=5.15
numpy>=1.26
scipy>=1.11
matplotlib>=3.8
scikit-learn>=1.3
scikit-image>=0.22
joblib>=1.3
psutil>=5.9
spectral>=0.23
h5py>=3.9
tifffile>=2023.9
netCDF4>=1.6
astropy>=5.3
zarr>=2.16
pyinstaller>=6.3
```

`requirements_tf.txt` (extra packages for the full build only):

```
tensorflow-cpu>=2.16
tf-keras>=2.16
keras-cv-attention-models>=1.4
```

Two choices matter here. The lower bounds (`>=`) keep the install flexible, and the packages are not pinned to exact versions; if you need a build you can repeat exactly, run `pip freeze` after a good build and keep that list. TensorFlow goes in its own file because it adds about 2 GB; `tensorflow-cpu` is used because the Windows build runs on the CPU.

Step 3: make three small edits to the app file

These make the packaged program behave well. Each is a few lines.

1. **No console in a windowed program.** A windowed `.exe` has no console, so `sys.stdout` and `sys.stderr` are `None` and any library that writes to them can crash. At the start of `main()` give them a sink:

```
for _stream in ("stdout", "stderr"):
    if getattr(sys, _stream) is None:
        setattr(sys, _stream, open(os.devnull, "w"))
```

2. **Know when you are packaged.** Near the top of the file add `FROZEN = bool(getattr(sys, "frozen", False))`. PyInstaller sets `sys.frozen` inside the `.exe`. Use it to change messages that tell people to `pip install tensorflow`, which makes no sense in a packaged program; in a packaged lite build say that the full build is needed.
3. **A self-test switch.** Add a function `run_selftest()` and call it from the entry point when `--selftest` is in `sys.argv`. It should:
   - start the window off-screen by setting `QT_QPA_PLATFORM=offscreen`,
   - build a small synthetic cube with a few known classes,
   - run the core steps (PCA, MNF, Random Forest and both SVMs) and check the accuracy is sensible,
   - import each optional library (`skimage`, `spectral`, `h5py`, `tifffile`, `joblib`, `scipy.io`) so a missing one is caught,
   - draw one matplotlib figure,
   - write one line per check, ending in `SELFTEST OK` or `SELFTEST FAILED: <names>`, to a log file (the program has no console) and exit with code 0 or 1.

Treat a missing TensorFlow as informational, not as a failure, so the same test works for both builds.

Also check that the multiprocessing line `multiprocessing.freeze_support()` is the first thing in `main()`. Without it, any library that starts worker processes can open extra copies of the window in a packaged program.

Step 4: write the PyInstaller spec file

The spec file `HyperVista.spec` is Python code that PyInstaller runs. You could build with one command line instead (`pyinstaller --windowed --icon ...`), but a spec keeps every decision in one place and can switch features on and off. Create it in the kit folder in these pieces.

A. Settings and switches. Read the folder, the script name, the icon, and two switches from environment variables, so one file serves the lite, full and debug builds:

```
import os, sys, importlib.util
from PyInstaller.utils.hooks import collect_all

HERE = os.path.abspath(SPECPATH)
SCRIPT = os.path.join(HERE, "HyperVista_v1_0.py")
ICON = os.path.join(HERE, "ds3.ico")
WITH_TF = os.environ.get("HV_WITH_TF", "0") == "1"
CONSOLE = os.environ.get("HV_CONSOLE", "0") == "1"
```

B. A helper that checks whether a package is installed, so the spec never fails because an optional package is absent:

```
def installed(name):
    try:
        return importlib.util.find_spec(name) is not None
    except Exception:
        return False
```

C. Data files. Files the program reads at run time must be copied into the package. The tuple means (source file, folder inside the package). The app looks for its icon, so ship it under both names:

```
datas = [(ICON, "."), (ICON, "assets"),
         (os.path.join(HERE, "hypervista.ico"), ".")]
binaries, hiddenimports = [], []
```

D. Hidden imports. PyInstaller finds modules by reading `import` statements. It cannot see modules the app loads by a string name, such as `importlib.import_module("h5py")`. List those and use `collect_all`, which gathers a package's code, data files and compiled libraries:

```
optional = ["h5py", "tifffile", "netCDF4", "cftime", "zarr", "astropy",
            "spectral", "rasterio", "scipy.io", "joblib", "psutil",
            "skimage", "sklearn", "imagecodecs"]
for name in optional:
    if installed(name):
        d, b, h = collect_all(name)
        datas += d; binaries += b; hiddenimports += h
        hiddenimports.append(name)

hiddenimports += ["PyQt5.sip", "matplotlib.backends.backend_qt5agg",
                  "matplotlib.backends.backend_agg"]
```

E. Excludes and the TensorFlow switch. Leave out libraries the app never uses; this keeps the folder smaller. TensorFlow is added only when the switch is on:

```
excludes = ["tkinter", "PyQt6", "PySide2", "PySide6", "IPython",
            "jupyter", "notebook", "pytest", "torch", "tensorboard"]
if WITH_TF:
    for name in ("tensorflow", "keras", "tf_keras",
                 "keras_cv_attention_models"):
        if installed(name):
            d, b, h = collect_all(name)
            datas += d; binaries += b; hiddenimports += h
else:
    excludes += ["tensorflow", "keras", "tf_keras",
                 "keras_cv_attention_models"]
```

F. The three build stages. Analysis follows the imports, PYZ packs the Python code, EXE makes the launcher and COLLECT assembles the output folder:

```
a = Analysis([SCRIPT], pathex=[HERE], binaries=binaries, datas=datas,
             hiddenimports=hiddenimports, excludes=excludes)
pyz = PYZ(a.pure)
exe = EXE(pyz, a.scripts, [], exclude_binaries=True, name="HyperVista",
          upx=False, console=CONSOLE,
          icon=ICON if sys.platform.startswith("win") else None)
coll = COLLECT(exe, a.binaries, a.datas, upx=False, name="HyperVista")
```

Three details to know:

- **`exclude_binaries=True` plus `COLLECT`** is what makes the output a folder with `HyperVista.exe` and `_internal`. Leaving them out would make a single-file `.exe` that unpacks itself to a temporary folder on every start (slow, and more often flagged by antivirus).
- **`console=False`** gives a normal windowed program. The debug build sets it to `True`.
- **`upx=False`** skips compression of the libraries, which avoids broken Qt and TensorFlow DLLs and many antivirus false alarms.

How to find what a new app needs

When you change the app and the packaged program fails with `ModuleNotFoundError: No module named 'xyz'`, add `xyz` to the `optional` list (or to `hiddenimports`) and rebuild. Run the debug build to see the message. The self-test is the quickest way to catch this early.

Step 5: write the one-click script

`build_windows.bat` runs the whole build so nobody has to type commands. Save it as plain text with the `.bat` extension:

```
@echo off
setlocal
cd /d "%~dp0"
where py >nul 2>nul || (echo Python launcher not found. Install Python 3.11.
& pause & exit /b 1)
if not exist .venv\Scripts\python.exe py -3.11 -m venv .venv || (echo Could
not create the environment. & pause & exit /b 1)
call .venv\Scripts\activate.bat
python -m pip install --upgrade pip || goto :fail
pip install -r requirements.txt || goto :fail
set HV_WITH_TF=0
set HV_CONSOLE=0
if /i "%1"=="full" (
    pip install -r requirements_tf.txt || goto :fail
    set HV_WITH_TF=1
)
if /i "%1"=="debug" set HV_CONSOLE=1
pyinstaller --noconfirm --clean HyperVista.spec || goto :fail
dist\HyperVista\HyperVista.exe --selftest "%~dp0selftest.log"
if errorlevel 1 (echo Self-test reported a problem - see selftest.log) else
(echo Self-test passed.)
```

```
echo Done: %~dp0dist\HyperVista\HyperVista.exe
pause
exit /b 0
:fail
echo BUILD FAILED - see the messages above.
pause
exit /b 1
```

What each part is for:

- `cd /d "%~dp0"` makes the script work from the folder it sits in, however it was started.
- `where py` checks that the Python launcher exists before anything else.
- `py -3.11 -m venv .venv` makes a private environment, so the build never touches other Python projects. Change `3.11` here (and nowhere else) to use a different Python.
- `|| goto :fail` stops the script at the first error instead of carrying on with a broken environment.
- The `set HV_...` lines pass the lite, full or debug choice to the spec file from Step 4.
- `--clean` makes PyInstaller discard its cache, so a stale build cannot hide a problem.
- The self-test line runs the finished `.exe`, so a build that packaged badly is caught at once.

Step 6: the optional installer script

Inno Setup (free, jrsoftware.org) turns the `dist\HyperVista` folder into one `HyperVista_Setup.exe`. Save this as `installer.iss`:

```
[Setup]
AppName=HyperVista (DS3)
AppVersion=1.0
AppPublisher=Kapih Team
DefaultDirName={autopf}\HyperVista
DefaultGroupName=HyperVista
OutputBaseFilename=HyperVista_Setup
OutputDir=installer_out
SetupIconFile=ds3.ico
UninstallDisplayIcon={app}\HyperVista.exe
Compression=lzma2
SolidCompression=yes
```

```
ArchitecturesInstallIn64BitMode=x64compatible
PrivilegesRequired=lowest

[Files]
Source: "dist\HyperVista\*"; DestDir: "{app}"; Flags: recursesubdirs
createallsubdirs ignoreversion

[Icons]
Name: "{group}\HyperVista"; Filename: "{app}\HyperVista.exe"
Name: "{autodesktop}\HyperVista"; Filename: "{app}\HyperVista.exe"; Tasks:
desktopicon

[Tasks]
Name: "desktopicon"; Description: "Create a desktop shortcut"; Flags:
unchecked

[Run]
Filename: "{app}\HyperVista.exe"; Description: "Start HyperVista"; Flags:
nowait postinstall skipifsilent
```

`PrivilegesRequired=lowest` lets people install without administrator rights. Change `AppVersion` for each release.

Step 7: test the kit before you hand it over

1. On any machine, run the build. On Windows use `build_windows.bat`; on Linux the same spec works with `pyinstaller HyperVista.spec`, which is how this kit was checked in lite and full mode.
2. Run `HyperVista.exe --selftest selftest.log` and read the log. Every line should say `OK`, except `TensorFlow bundled` in the lite build.
3. Start the program normally and open each tab once.
4. Test on a second Windows PC that has no Python, which is the real user's situation.

Step 8: zip the kit

1. Delete the temporary folders: `.venv`, `build`, `dist`, `installer_out` and any `__pycache__`. The kit should hold only the source files.
2. Right-click the kit folder and choose Send to, then Compressed (zipped) folder. The result is `HyperVista_build.zip`, about 250 KB.

3. Keep the Python version and the date of the build in the file name or a note (for example Python 3.11, November 2026), because a later Python release may need small changes.

Preparing the Windows PC

You need a 64-bit Windows 10 or 11 PC, an internet connection for the first build, and about 10 GB of free disk space for the lite build (more for the full one).

Install Python 3.11

1. Download a Python 3.11 installer for Windows (64-bit) from python.org. Binary installers stop at **3.11.9**; the newer 3.11.x pages are source-only and have no installer. Your installed 3.11.0 also works.
2. On the first installer screen tick **Add python.exe to PATH** and keep the **py launcher** option ticked, then choose Install Now. A shortcut: `winget install Python.Python.3.11` in Command Prompt.
3. Open a new Command Prompt and check:

```
py -3.11 --version
```

It should print `Python 3.11.x`.

Why not Python 3.15

A brand-new Python release usually lacks prebuilt packages for numpy, scipy, scikit-learn, scikit-image, h5py and especially TensorFlow, so the install step fails or tries to compile from source. The build script asks for 3.11 explicitly with `py -3.11`, so you can keep 3.15 installed next to it; nothing else on the PC is changed. Python 3.12 is the fallback: change both `py -3.11` entries in `build_windows.bat` to `py -3.12`.

Unpack the kit

1. Right-click `HyperVista_build.zip` and choose Extract All.
2. Extract to a short path such as `C:\HyperVista_build`. Avoid long paths and cloud-synced folders such as OneDrive.
3. Check that the folder holds `HyperVista_v1_0.py`, `HyperVista.spec`, `build_windows.bat`, `requirements.txt`, `requirements_tf.txt` and the two `.ico` files.

Building the .exe

Open Command Prompt inside the kit folder (type `cmd` in the Explorer address bar and press Enter) and run one of these:

```
build_windows.bat
build_windows.bat full
build_windows.bat debug
```

The first build takes about 10 to 25 minutes because it downloads and installs the packages. Later builds are faster.

What the script does

1. **Checks for Python.** It looks for the `py` launcher and stops with a message if it is missing.
2. **Creates `.venv`.** A private Python 3.11 environment inside the kit folder, so your other Python projects are untouched.
3. **Updates pip and installs `requirements.txt`.** For the full build it also installs `requirements_tf.txt` and sets `HV_WITH_TF=1`. For the debug build it sets `HV_CONSOLE=1`.
4. **Runs PyInstaller** with `HyperVista.spec`. PyInstaller reads the app, follows its imports, adds the hidden modules and data files from the spec, and writes `dist\HyperVista\` containing `HyperVista.exe` and `_internal`.
5. **Runs the self-test** with `HyperVista.exe --selftest selftest.log`, then reports whether it passed.
6. **Prints the result path** and reminds you to keep the `_internal` folder next to the `.exe`.

What success looks like

```
Self-test passed.
Done:  C:\HyperVista_build\dist\HyperVista\HyperVista.exe
```

In the lite build `selftest.log` also contains `FAIL TensorFlow bundled`. That line is expected and does not mean the build failed; the verdict is the last line, `SELFTEST OK`.

Folders the build creates

Folder	Content	Safe to delete after the build?
`.venv`	The private Python environment	Yes, but the next build then reinstalls everything
`build`	PyInstaller working files	Yes
`dist\HyperVista`	The finished program	No, this is the result

Running, sharing and installing

Run and check

1. Double-click `dist\HyperVista\HyperVista.exe`. The window should open with the HyperVista tabs.
2. Load a small cube and try one step, for example the Dimensionality Reduction tab or Classification with Random Forest.
3. In the lite build, the Segmentation tab shows a message that the deep-learning models need the full build. That is correct.

Share the program

1. Zip the whole `dist\HyperVista` folder. The `.exe` does not work without the `_internal` folder next to it.
2. The receiver extracts the zip, opens the `HyperVista` folder and runs `HyperVista.exe`. They need no Python.
3. Windows SmartScreen may say "unknown publisher". They click **More info**, then **Run anyway**. A code-signing certificate removes this warning.

Optional: one installer file

1. Install **Inno Setup 6** from jrsoftware.org.
2. Open `installer.iss` in Inno Setup and choose **Build, then Compile**.
3. You get `installer_out\HyperVista_Setup.exe`, which installs per user, adds a Start-menu entry and an optional desktop shortcut, and adds an uninstaller.

Alternative: let GitHub build it

If you do not want to set up Python on a Windows PC, the kit includes a GitHub workflow.

1. Create a GitHub repository and upload the contents of the kit, keeping the `.github` folder.
2. Open **Actions, then Build Windows app, then Run workflow**, pick lite or full and a version label such as `v1.0.1`. After about 15 to 30 minutes the zip appears under Artifacts.
3. For a public download, push a tag (`git tag v1.0.1` then `git push origin v1.0.1`). The workflow attaches `HyperVista_Windows_v1.0.1.zip` to a GitHub Release, which users download, extract and run.
4. GitHub limits each release file to 2 GB. The lite zip is far below that; the full zip may come close, so share it as a workflow artifact or from a file share.

Troubleshooting, updates and limits

Troubleshooting

Symptom	Likely cause	Fix
`py` is not recognised	Python launcher not installed or PATH not set	Reinstall Python 3.11 with Add to PATH and the py launcher ticked, then open a new Command Prompt
Script says it cannot create the environment	Python 3.11 is not installed	Install 3.11 (3.11.9 or the 3.11.0 you have) and check `py -3.11 --version`
`pip install` fails with `No matching distribution` or `Failed building wheel`	Python version too new (for example 3.15)	Build with 3.11, or 3.12 after editing the two `py -3.11` entries in the script
`pip install` cannot connect	Network, firewall or proxy	Check the connection; behind a proxy set `HTTPS_PROXY` first
The program opens and closes at once	A runtime error hidden by the windowless mode	Rebuild with `build_windows.bat debug` and read the console

Symptom	Likely cause	Fix
Antivirus deletes or blocks the build	False alarm common for PyInstaller programs	Add an exclusion for the kit folder, then rebuild; signing the build avoids it
SmartScreen shows "unknown publisher"	The program is not code-signed	More info, then Run anyway
`HyperVista.exe` does nothing when copied alone	The `_internal` folder was left behind	Move or zip the whole `dist\HyperVista` folder
Segmentation says TensorFlow is missing	Lite build	Use `build_windows.bat full`
Full build fails or runs out of memory	Too little free RAM or disk, or wrong Python version	Close other programs, keep at least 8 GB of RAM free and 15 GB of disk, use Python 3.11
Self-test reports `FAILED` and a name	A library did not bundle	Rebuild with `debug`; send the line from `selftest.log` and the last lines of the build output

Updating the app later

1. Replace `HyperVista_v1_0.py` in the kit folder with the new file. Keep the three build additions (the self-test, the `FROZEN` messages and the console guard), or ask for a rebuilt kit file.
2. If the new version needs a new package, add it to `requirements.txt`; if it loads it by name at run time, add it to the `optional` list in `HyperVista.spec` as well.
3. Run `build_windows.bat` again. Delete the `build` and `dist` folders first if you want a completely clean build.

Limits

- The Windows build has not been run by me. The recipe was built and self-tested on Linux in lite and full mode, so the first Windows build is the real test.
- The packaged program runs the deep-learning models on the CPU only; TensorFlow has no native Windows GPU support after version 2.10.
- The lite build cannot train the 16 segmentation models. Use the full build for that.

- The program is not code-signed, so Windows and some antivirus tools may warn on first run.